

Sesjon 3 — Oppgaver

Blokk 1: Jinja og templating

OPPGAVER

1. Les kompilert SQL

Kjør `dbt compile` på en av staging-modellene dine og åpne den kompilerte filen:

```
dbt compile --select stg__crm_customers
```

Finn filen i `target/compiled/` og les den.

Spørsmål å tenke på:

- Hva ble `{{ source('crm', 'crm_customers') }}` til?
- Hva ble `{{ ref('stg__crm_customers') }}` til i en modell som bruker den?
- Er det noe i den kompilerte SQL-en du ikke forventet?

2. Dev-grense med `target.name`

Legg til en `LIMIT`-klausul i en av intermediate-modellene som bare er aktiv i `dev`:

```
from {{ ref('stg_crm__customers') }}

{% if target.name == 'dev' %}
limit 500
{% endif %}
```

Kjør `dbt compile --select <modell>` og bekreft at `LIMIT` vises i kompilert SQL.

Kjør `dbt run --select <modell>` og sjekk radantall i databasen.

Hva er verdien av dette mønsteret? Hva er risikoen hvis det brukes ukritisk?

3. Variabel og betingelse i SQL

`billing_invoices` har krediterte fakturaer med negativ `amount_eur`. Bruk en Jinja-variabel og betingelse for å filtrere dem inn eller ut avhengig av målgruppe:

```
{% set inkluder_krediterte = true %}

select *
from {{ ref('stg__billing_invoices') }}
{% if not inkluder_krediterte %}
where status != 'credited'
{% endif %}
```

Kompiler med begge verdiene av `inkluder_krediterte` og les den kompilerte SQL-en.

Diskuter: Er dette en god bruk av Jinja, eller finnes det en bedre løsning?

4. Løkke over statusverdier

`crm_contracts` har tre mulige statusverdier: `active`, `suspended`, `terminated`.

Skriv en SQL-spørring som bruker `{% for %}` til å generere én kolonne per status:

```
{% set statuser = ['active', 'suspended', 'terminated'] %}

select
  customer_id
  {% for s in statuser %}
    , sum(case when status = '{{ s }}' then 1 else 0 end) as {{ s }}_contracts
  {% endfor %}
from {{ ref('stg__crm_contracts') }}
group by 1
```

Lagre dette som en analyse i `analyses/contract_status_pivot.sql` og kompiler den.

DISKUSJON

A. Når er Jinja en forbedring?

`{% for s in statuser %}` og hardkodede CASE-setninger produserer identisk SQL. Hva er det reelle argumentet for Jinja-varianten? Når blir det en ulempe — for eksempel når du leser en diff i en pull request?

B. `target.name`-betingelser og testbarhet

Kode som oppfører seg ulikt i dev og prod er vanskeligere å teste. Hva er alternativene til `{% if target.name == 'dev' %}`
`limit 500 {% endif %}`? Hvilke mønstre bruker din organisasjon?

C. Synlighet i kompilert SQL

dbt-kjøring viser ikke Jinja — bare den kompilerte SQL-en. Hvem i teamet ditt har behov for å forstå Jinja, og hvem trenger kun å forstå kompilert SQL? Påvirker det hvordan du skriver og dokumenterer makroer?

D. Filtre og `| trim`

`generate_schema_name`-makroen bruker `{{ custom_schema_name | trim }}`. Prøv å fjerne `| trim` og observer hva som skjer med skjemanavn når `dbt run` kjøres. Hva slags feil ville dette føre til i produksjon?

EKSTRAOPPGAVER

E1. `{{ log() }}` — feilsøk Jinja

dbt har en innebygd `log()`-funksjon som printer til terminalen under kompilering — nyttig for å forstå hva Jinja-variabler inneholder.

```
{% set statuser = ['active', 'suspended', 'terminated'] %}  
{{ log("Antall statuser: " ~ statuser | length, info=true) }}
```

Legg til `log()`-kall i en analyse og kjør `dbt compile`. Bruk det til å inspisere innholdet i `target`-objektet:

```
{{ log(target | tojson, info=true) }}
```

Hva inneholder `target`? Hva er forskjellen mellom `info=true` og `info=false`?

E2. Jinja i YAML — vars

dbt støtter prosjektvariabler definert i `dbt_project.yml` og referert med `{{ var() }}` i SQL og YAML:

```
# dbt_project.yml
vars:
  min_kontrakt_dato: '2020-01-01'
```

```
where start_date >= '{{ var("min_kontrakt_dato") }}'
```

Definer én variabel som er relevant for Televerket-prosjektet, bruk den i en modell, og overstyr den fra kommandolinjen:

```
dbt run --vars '{"min_kontrakt_dato": "2022-01-01"}'
```

Se [dbt-dokumentasjonen](#) for detaljer.

Blokk 2: Makroer

OPPGAVER

1. Skriv `parse_yyyymmdd`

Opprett `macros/parse_yyyymmdd.sql` med en enkel implementasjon for DuckDB:

```
{% macro parse_yyyymmdd(column_name) %}  
    strptime({{ column_name }}, '%Y%m%d')::date  
{% endmacro %}
```

Bruk makroen i `stg__billing_invoices` for begge datokolonnene:

```
{{ parse_yyyymmdd('invoice_date') }} as invoice_date,  
{{ parse_yyyymmdd('due_date') }}      as due_date,
```

Kjør `dbt compile --select stg__billing_invoices` og bekreft at makroen ekspanderes riktig.

Kjør `dbt run --select stg__billing_invoices` og verifiser at datoene ser riktige ut i databasen.

2. Legg til standardverdi for format

Utvid makroen med en valgfri `format`-parameter:

```
{% macro parse_yyyymmdd(column_name, format='%Y%m%d') %}  
    strptime('{{ column_name }}', '{{ format }}')::date  
{% endmacro %}
```

Bekreft at eksisterende kall fortsatt fungerer uten å sende inn format.

Tenk igjenom: Finnes det andre datokolonner i kildedataene som bruker et annet format? Ville de nytt godt av denne makroen med overstyrt format?

3. Legg til `adapter.dispatch` for flerdatabasestøtte

Bygg om makroen til å støtte BigQuery og Snowflake i tillegg til DuckDB:

```
{% macro parse_yyyymmdd(column_name) %}  
  {{ return(adapter.dispatch('parse_yyyymmdd')(column_name)) }}  
{% endmacro %}  
  
{% macro default__parse_yyyymmdd(column_name) %}  
  to_date('{{ column_name }}', 'YYYYMMDD')  
{% endmacro %}  
  
{% macro duckdb__parse_yyyymmdd(column_name) %}  
  strptime('{{ column_name }}', '%Y%m%d')::date  
{% endmacro %}  
  
{% macro bigquery__parse_yyyymmdd(column_name) %}  
  parse_date('%Y%m%d', '{{ column_name }}')  
{% endmacro %}
```

Kjør `dbt compile --select stg__billing_invoices` og se at `duckdb__parse_yyyymmdd` velges lokalt.

4. Skriv en generisk test

Fakturabeløp skal alltid være positive — med unntak av krediterte fakturaer. Skriv en generisk test som sjekker dette:

```
-- macros/test_is_positive_or_credited.sql
{% macro test_is_positive_or_credited(model, column_name) %}
    select *
    from {{ model }}
    where {{ column_name }} < 0
        and status != 'credited'
{% endmacro %}
```

Legg testen til i `_staging.yml`:

```
- name: amount_eur
  tests:
    - is_positive_or_credited
```

Kjør `dbt test --select stg__billing_invoices` og bekreft at testen passerer.

DISKUSJON

A. Makro eller CTE?

`parse_yyyymmdd` brukes to steder i `stg__billing_invoices` og potensielt nullsteder andre steder i dette prosjektet. Ville en CTE inni modellen løst det samme? Hva er det konkrete argumentet for en makro her kontra bare å gjenta `strptime(col, '%Y%m%d')::date` inline?

B. Navngiving og navnerom

`parse_yyyymmdd` er et globalt navn i dbt — ingen navnerom skiller det fra pakker. Hvis `dbt_utils` hadde en makro med samme navn, ville de kollidere. Hva er konvensjonen for å unngå dette? Se på hvordan `dbt_utils` navngir sine makroer.

C. Dokumentasjon av makroer

dbt støtter dokumentasjon av makroer i `schema.yml`. Se [dbt-dokumentasjonen](#). Bør makroer dokumenteres like grundig som modeller? Hvem er målgruppen for makro-dokumentasjon?

D. Generisk test vs. `dbt_utils.expression_is_true`

`test_is_positive_or_credited` kan alternativt skrives som:

```
- dbt_utils.expression_is_true:
  expression: ">= 0 or status = 'credited'"
```

Hva er forskjellen i praksis? Når gir det mening å skrive en egendefinert generisk test fremfor å bruke `expression_is_true`?

EKSTRAOPPGAVER

E3. Makro for JSON-utpakking

`preferences`-kolonnen i `crm_customers` og `device_info` i `network_service_activations` bruker begge JSON-utpakking. Mønsteret gjentas på tvers av modellene.

Skriv en makro `json_extract(column_name, path)` som pakker inn database-spesifikk JSON-syntaks:

- DuckDB: `json_extract_string({{ column_name }}, '{{ path }}')`
- BigQuery: `json_value({{ column_name }}, '{{ path }}')`
- Snowflake: `{{ column_name }}:{{ path }}::string`

Bruk makroen i én av intermediate-modellene som pakker ut JSON-felt.

E4. Utforsk `generate_schema_name`

`macros/generate_schema_name.sql` overstyrer dbt sin standardlogikk. Les makroen nøye og svar på:

- Hva skjer med seeds?
- Hva skjer med modeller uten `custom_schema_name`?
- Hva er forskjellen mellom `prod`-kjøring og alle andre?

Endre `target.name == 'prod'` til `target.name == 'dev'` og kjør `dbt run`. Hva endrer seg i databasen?

Tilbakestill endringen og diskuter: Hva ville konsekvensen vært av å deploye denne endringen til produksjon ved en feiltakelse?

Blokk 3: dbt-pakke-økosystemet

OPPGAVER

1. Opprett `packages.yml` og installer pakker

Opprett `packages.yml` i roten av prosjektet:

```
packages:  
  - package: dbt-labs/dbt_utils  
    version: 1.3.0  
  - package: dbt-labs/dbt_codegen  
    version: 0.13.1  
  - package: dbt-labs/dbt_date  
    version: 0.10.2  
  - package: dbt-labs/audit_helper  
    version: 0.12.0
```

```
dbt deps
```

Verifiser at `dbt_packages/`-mappen ble opprettet. Legg `dbt_packages/` til i `.gitignore` hvis det ikke allerede er der.

2. Legg til `dbt_utils`-tester

Legg til minst to `dbt_utils`-tester i eksisterende YAML-filer:

```
# Eksempel 1 – due_date må være etter invoice_date
- name: due_date
  tests:
    - dbt_utils.expression_is_true:
        expression: ">= invoice_date"

# Eksempel 2 – kombinasjon av kolonner er unik
models:
  - name: int_billing__invoice_settlement
    tests:
      - dbt_utils.unique_combination_of_columns:
          combination_of_columns:
            - invoice_id
            - customer_id
```

Kjør `dbt test` og observer outputen. Passerer alle testene?

3. Generer YAML med `dbt_codegen`

Bruk `generate_model_yaml` til å generere YAML-skjelett for en modell som mangler fullstendig dokumentasjon:

```
dbt run-operation generate_model_yaml \
  --args '{"model_names": ["int_customers__enriched"]}'
```

Kopier outputen inn i `_intermediate.yml`. Sammenlign med det som allerede er der — hva har du lagt til manuelt som `generate_model_yaml` ikke genererer?

Mål: fullstendig kolonnedekning med beskrivelser på `int_customers__enriched`.

4. Generer source-YAML med `dbt_codegen`

Bruk `generate_source` til å generere YAML for en kildetabell fra bunnen av:

```
dbt run-operation generate_source \  
  --args '{"schema_name": "raw", "table_names": ["network_incidents"]}'
```

Sammenlign outputen med det som allerede ligger i `__sources.yml`. Fanger `generate_source` opp kolonner som ikke er dokumentert ennå?

5. Sett opp elementary

Legg `elementary-data/elementary` til i `packages.yml` og kjør `dbt deps`.

Konfigurer i `dbt_project.yml`:

```
models:
  elementary:
    +schema: elementary
    +materialized: incremental
```

```
dbt run --select elementary
```

Verifiser at elementary-tabellene ble opprettet i databasen under `elementary`-schema.

6. Legg til elementary-tester og generer rapport

Legg til anomalitester på én modell:

```
models:
  - name: stg__billing_invoices
    tests:
      - elementary.volume_anomalies:
          timestamp_column: invoice_date
    columns:
      - name: amount_eur
        tests:
          - elementary.column_anomalies:
              column_anomalies:
                - null_count
                - average
```

```
pip install 'elementary-data[duckdb]'
dbt test --select stg__billing_invoices
edr report
```

Åpne rapporten i nettleseren. Hva viser den etter kun én kjøring? Hva mangler det for at anomalideteksjon skal være meningsfull?

DISKUSJON

A. Pakkeierskap og governance

Hvem i teamet bør bestemme hvilke pakker som legges til et prosjekt? Hva er risikoen ved at enkeltpersoner legger til pakker fritt? Bør pakker gjennomgås i pull requests på samme måte som kode? Finnes det en fornuftig «godkjent pakkeliste» i din organisasjon?

B. Skrive selv vs. bruke pakke

`parse_yyyymmdd` er en egenskrevet makro. `dbt_utils.expression_is_true` er en pakkemakro. Hva er kriteriene for å velge det ene fremfor det andre? Hva er kostnaden ved å ta inn en pakkedependency for én makro du egentlig kunne skrevet på tre linjer selv?

C. elementary vs. statiske terskler

elementary lærer hva som er normalt og varsler ved avvik. Statiske terskler (`warn_after: 24 timer` , `expression_is_true: amount_eur >= 0`) feiler ved et kjent brudd. Hvilke typer datakvalitetsproblemer i Televerket-datasettet passer best for hvert alternativ? Er de komplementære eller konkurrerende?

D. `dbt_codegen` og eierskap til YAML

`generate_model_yaml` genererer et skjelett. Beskrivelser og tester må legges til manuelt. Hva er risikoen ved å regenerere YAML for en modell som allerede har manuelt skrevet innhold? Bør `dbt_codegen` kun brukes ved første gangs oppsett, eller er det nyttig løpende?

EKSTRAOPPGAVER

E5. `dbt_utils.star()` i en mart-modell

Lag en enkel mart-modell `mart_customer_360.sql` som baserer seg på `int_customers__enriched`. Bruk `dbt_utils.star()` til å eksponere alle kolonner unntatt interne metadata-kolonner:

```
select
  {{ dbt_utils.star(
    from=ref('int_customers__enriched'),
    except=['_loaded_at']
  ) }}
from {{ ref('int_customers__enriched') }}
```

Kompiler og les den genererte SQL-en. Hva skjer i den kompilerte filen hvis du legger til en ny kolonne i `int_customers__enriched`?

E6. `audit_helper` — sammenlign to modellversjoner

`audit_helper` lar deg sammenligne to versjoner av en modell rad for rad — nyttig etter refaktorering.

Se [audit_helper-dokumentasjonen](#).

Gjør en liten endring i en av staging-modellene dine (f.eks. legg til eller fjern en kolonne), og bruk

`audit_helper.compare_relations` til å se differansen:

```
-- analyses/compare_stg_invoices.sql
{{ audit_helper.compare_relations(
    a_relation=ref('stg__billing_invoices'),
    b_relation=ref('stg__billing_invoices__v2')
) }}
```

Diskuter: Når er dette nyttig i en produksjonspipeline? Hva er alternativet til `audit_helper` for å validere at en refaktorering ikke endrer data?

E7. Makrodokumentasjon i YAML

dbt støtter dokumentasjon av makroer i en egen `schema.yml`-fil under `macros/`.

Se [dbt-dokumentasjonen](#).

Opprett `macros/schema.yml` og dokumenter `parse_yyyymmdd` og den generiske testen du skrev:

```
macros:
  - name: parse_yyyymmdd
    description: >
      Parser en YYYYMMDD-formatert streng til DATE.
      Støtter DuckDB, BigQuery og Snowflake via adapter.dispatch.
    arguments:
      - name: column_name
        type: string
        description: Kolonnen som skal parses.
```

Generer docs og sjekk at makroen dukker opp i dokumentasjonssiden:

```
dbt docs generate && dbt docs serve
```